Full-Stack Engineering Training — Day 12 Study Material

MODULE	TOPIC	PORT & STACK
Backend Engineering (NestJS)	DTOs, Class-Validator & Global Error Handling	Port 3012 • ValidationPipe

Key Learning Objectives: Understand the role of Data Transfer Objects (DTOs) in modern enterprise APIs, declarative validation using `class-validator` and `class-transformer`, mass assignment protection with `whitelist: true` and `forbidNonWhitelisted: true`, nested DTO validation with `@ValidateNested()`, and standardizing application error formats with custom `AppException` and global `AllExceptionsFilter`.

1. What is a DTO and Why Do We Need It?

A **Data Transfer Object (DTO)** is an object that defines how data is transmitted over the network. It establishes a strict input/output contract before data reaches any controller logic or service layer.

> **DTO vs Database Entity:**
> - **DTO**: Shapes client input (e.g., passwords in plain text, user-submitted tower count). Handles validation rules and sanitization.
> - **Entity**: Shapes database storage (e.g., hashed passwords, relations, primary key UUIDs, timestamps).

2. Declarative Validation with class-validator

Rather than writing repetitive `if (!req.body.name || req.body.name.length < 3)` checks, NestJS uses TypeScript decorators from `class-validator`:

```
export class CreateProjectDto {
  @IsString({ message: 'Project name must be a string' })
  @Transform(({ value }) => typeof value === 'string' ? value.trim() : value)
  @MinLength(3, { message: 'Project name must be at least 3 characters' })
  @MaxLength(100)
  name: string;

  @IsEnum(ProjectStatus, { message: 'status must be: PLANNING, ACTIVE, COMPLETED, ON_HOLD' })
  status: ProjectStatus;

  @IsInt()
  @Min(1)
  @Max(50)
  towerCount: number;
}
```

3. Validation Pipe & Mass Assignment Protection

In `main.ts`, we register `ValidationPipe` with enterprise security flags:

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,            // Strips any property without a DTO decorator
    forbidNonWhitelisted: true, // Throws 400 Bad Request if extra fields are passed
    transform: true             // Automatically transforms request body to class instance
  })
);
```

Incoming Request Payload	Validation Pipe Behavior	HTTP Status
`{ name: "Apex", status: "ACTIVE", towerCount: 4 }`	All rules match, trimmed and sanitized	**201 Created**
`{ name: "Apex", status: "ACTIVE", role: "ADMIN" }`	"property role should not exist" (forbidNonWhitelisted)	**400 Bad Request**
`{ name: "AB", status: "ACTIVE", towerCount: 4 }`	"Project name must be at least 3 characters"	**400 Bad Request**
`{ name: "Apex", status: "PENDING", towerCount: 4 }`	"status must be one of: PLANNING, ACTIVE, COMPLETED, ON_HOLD"	**400 Bad Request**
`{ name: "Apex", status: "ACTIVE", towerCount: 99 }`	"Tower count cannot exceed 50"	**400 Bad Request**

4. Nested Object Validation with @ValidateNested()

For complex models like a Tower having an array of Floors, NestJS requires both `@ValidateNested()` and `@Type()` from `class-transformer`:

```
export class TowerDto {
  @IsString()
  @MinLength(2)
  name: string;

  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => FloorDto)
```

```
  floors: FloorDto[];
}
```

5. Global Exception Filter & Unified Error Architecture

All uncaught exceptions and `HttpException` instances are transformed into a clean, unified JSON schema:

```
{
  "statusCode": 404,
  "message": "Project with ID proj-999 was not found",
  "error": "NotFoundException",
  "timestamp": "2026-10-14T02:00:00.000Z",
  "path": "/projects/proj-999"
}
```

> **Verification & Automated Testing:**
> - All 8 unit and integration tests passed cleanly in `test-day12.cjs` .
> - Postman collection in `day12/postman_collection.json` contains both positive and negative validation tests.